Exam AWS Certified Developer - Associate DVA-C02 All Questions

View all questions & answers for the AWS Certified Developer - Associate DVA-C02 exam

Go to Exam

EXAM AWS CERTIFIED DEVELOPER - ASSOCIATE DVA-C02 TOPIC 1 QUESTIO...

Exam question from Amazon's AWS Certified Developer - Associate DVA-C02
Question #: 20
Topic #: 1
[All AWS Certified Developer - Associate DVA-C02 Questions]

A developer wants to store information about movies. Each movie has a title, release year, and genre. The movie information also can include additional properties about the cast and production crew. This additional information is inconsistent across movies. For example, one movie might have an assistant director, and another movie might have an animal trainer.
The developer needs to implement a solution to support the following use cases:
For a given title and release year, get all details about the movie that has that title and release year.
For a given title, get all details about all movies that have that title.
For a given genre, get all details about all movies in that genre.
Which data store configuration will meet these requirements?

A. Create an Amazon DynamoDB table. Configure the table with a primary key that consists of the title as the partition key and the release year as the sort key. Create a global secondary index that uses the genre as the partition key and the title as the sort key.

B. Create an Amazon DynamoDB table. Configure the table with a primary key that consists of the genre as the partition key and the release year as the sort key. Create a global secondary index that uses the title as the partition key.

C. On an Amazon RDS DB instance, create a table that contains columns for title, release year, and genre. Configure the title as the primary key.

D. On an Amazon RDS DB instance, create a table where the primary key is the title and all other data is encoded into JSON format as one additional column.

Show Suggested Answer

by Untamables at *March 21, 2023, 1:30 p.m.*

Disclaimers:
- ExamTopics website is **not** related to, affiliated with, endorsed or authorized by Amazon.
- Trademarks, certification & product names are used for reference only and belong to Amazon.

Comments

Bibay Highly Voted 2 years, 11 months ago

Selected Answer: A

A. Create an Amazon DynamoDB table. Configure the table with a primary key that consists of the title as the partition key and the release year as the sort key. Create a global secondary index that uses the genre as the partition key and the title as the sort key.

This option is the best choice for the given requirements. By using DynamoDB, the developer can store the movie information in a flexible and scalable NoSQL database. The primary key can be set to the title and release year, allowing for efficient retrieval of information about a specific movie. The global secondary index can be created using the genre as the partition key, allowing for efficient retrieval of information about all movies in a specific genre. Additionally, the use of a NoSQL database like DynamoDB allows for the flexible storage of additional properties about the cast and crew, as each movie can have different properties without affecting the structure of the database.

upvoted 16 times

lebbar Most Recent 3 days, 2 hours ago

Selected Answer: A

Correct — Title (PK) + release year (SK) supports both "get by title+year" and "get all by title" queries. GSI with genre as PK covers "get all by genre". Flexible schema handles inconsistent cast/crew attributes natively in DynamoDB.

upvoted 1 times

sumanshu 1 year, 3 months ago

Selected Answer: A

B) Eliminated - No efficient way to query by title + release year since the primary key is genre + release year, and the GSI only has the title

C) Eliminated - Adding new columns for properties like assistant director or animal trainer becomes cumbersome and violates flexibility.

D) Eliminated - JSON in RDS is harder to query

upvoted 2 times

trieudo 1 year, 3 months ago

Selected Answer: A

keyword: (title + release year), title, genre, each film has differential cast and crew

=> Discard C,D: when it store film by structure && option C must scan all data that is not primary key(title), seem be not flexible && option D is horrible way, when storing data as json structure, take time to extract this json
==> discard B: querry (title + release year) must scan all item without combining them in primary key(partition key and sort key) at primary table or GSI

upvoted 2 times

ahadh7621 1 year, 8 months ago

Selected Answer: A

This question was on my exam July 23rd, 2024. Answer is A

upvoted 3 times

sid4510 1 year, 9 months ago

A is mostly correct , but I do see one problem there because in one year there can be same title movies can come which invalidate our primary key having title as partition key and year as a. Sort. key

upvoted 1 times

65703c1 1 year, 10 months ago

Selected Answer: A

A is the correct answer.

upvoted 1 times

leonardoliveros 2 years, 4 months ago

Selected Answer: A

If you create a primary key with title(pk) and release(sk) date you corvered two scenaries, and also you need a GSI by last scenary with genre so you should creating a GSI with genre (pk) and title (sk)

upvoted 2 times

Tony88 2 years, 7 months ago

Selected Answer: A

Go with A.
NoSQL is good when data attributes are inconsistent -> DynamoDB
Primary key should be unique, go with title + release year.

upvoted 4 times

jayvarma 2 years, 7 months ago

As the schema for each entry of data into the database is not the same all the time, We would require a NoSQL database. So, RDS DB instance is ruled out. The answer is between A and B.

As we would need the partition key to be as unique as possible, we would like to have the title of the movie as the partition key. Because having the partition key as the genre will create a hot partition problem and our data stored in the DynamoDB will be skewed.

So option A is the answer.

upvoted 4 times

Krok 2 years, 12 months ago

Selected Answer: A

It's A - I totally agree. It's a single appropriate solution. But in my opinion genre isn't a quite good option as GSI partition key - it isn't high distribution and we can get a hot partition.

upvoted 2 times

shahs10 3 years ago

Selected Answer: A

Option A because we have to search on the basis of title so it is better to partition by title. Also we have to search by genre so it is good option to make GSI using genre as partition key

upvoted 2 times

Untamables 3 years ago

Selected Answer: A

The correct answer is A.
Amazon DynamoDB is suited for storing inconsistent attributes data across items.
Option B is wrong. This solution does not help get items with the condition of the combination, title and release year.

upvoted 3 times